

Lab Exercise 3- Secure Data Transmission & Privacy – Encrypting PII Data at Rest and in Transit

1. Aim

To implement **encryption of Personally Identifiable Information (PII)** in a Java API to ensure secure data storage (**data at rest**) and secure data transfer (**data in transit**).

2. Software Requirements

- Java JDK 17
 - VS Code
 - Maven
 - Postman
-

3. Theory

Personally Identifiable Information (PII) refers to data that can identify a specific individual.

Examples of PII include:

- Name
- Email address
- Phone number
- Address
- Aadhaar / SSN

If PII is stored or transmitted without protection, it can lead to **data breaches and privacy violations**.

To protect PII, two important security techniques are used:

1. Encryption at Rest

Sensitive data stored in databases or files is **encrypted before storage**.

Example:

Email → encrypted before saving to database

2. Encryption in Transit

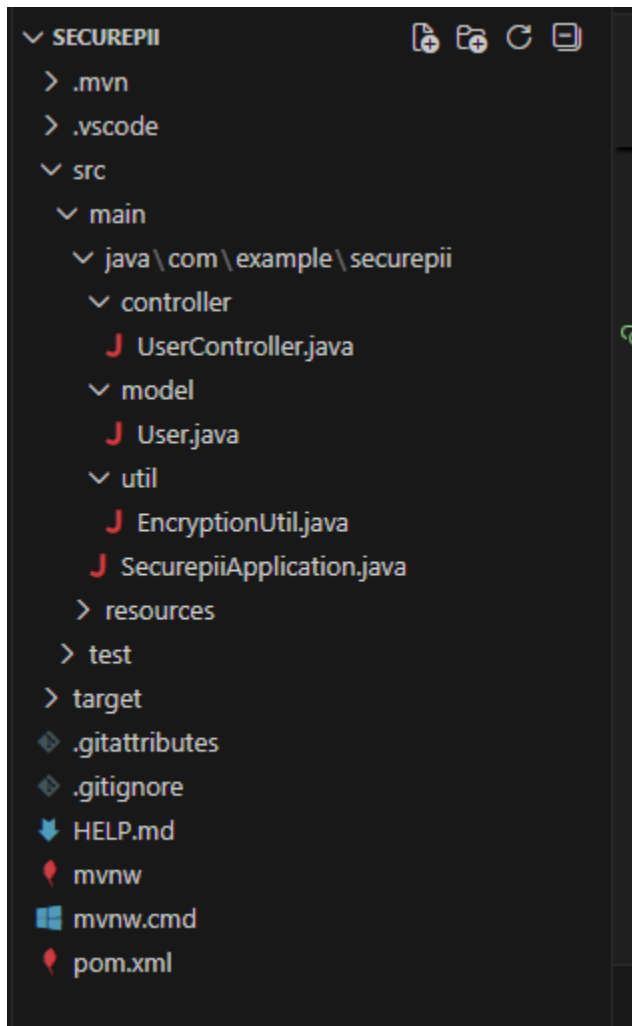
Data transmitted between client and server is encrypted using **HTTPS or encryption algorithms**.

Example:

Client → encrypted data → Server

In this experiment, **AES encryption** is used to secure PII data.

4. Project Directory Structure



5. Step 1: Create Spring Boot Project

Open **VS Code**

Press:

Ctrl + Shift + P

Select:

Spring Initializr: Create Maven Project

Project configuration:

Field	Value
Group	com.example
Artifact	securepii
Language	Java
Dependencies	Spring Web

6. Step 2: User Model (PII Data)

File:

model/User.java

```
package com.example.securepii.model;

public class User {

    private String name;
    private String email;
    private String phone;

    public String getName() {
        return name;
    }
}
```

```
}

public void setName(String name) {
    this.name = name;
}

public String getEmail() {
    return email;
}

public void setEmail(String email) {
    this.email = email;
}

public String getPhone() {
    return phone;
}

public void setPhone(String phone) {
    this.phone = phone;
}
}
```

Explanation

This class stores **PII data** such as:

- Name
 - Email
 - Phone number
-

7. Step 3: Encryption Utility

File:

```
util/EncryptionUtil.java
```

```
package com.example.securepii.util;

import javax.crypto.Cipher;
import javax.crypto.spec.SecretKeySpec;
import java.util.Base64;

public class EncryptionUtil {

    private static final String SECRET_KEY = "1234567890123456";

    public static String encrypt(String data) throws Exception {

        SecretKeySpec key = new SecretKeySpec(SECRET_KEY.getBytes(), "AES");

        Cipher cipher = Cipher.getInstance("AES");

        cipher.init(Cipher.ENCRYPT_MODE, key);

        byte[] encrypted = cipher.doFinal(data.getBytes());

        return Base64.getEncoder().encodeToString(encrypted);
    }

    public static String decrypt(String encryptedData) throws Exception {
```

```
SecretKeySpec key = new SecretKeySpec(SECRET_KEY.getBytes(), "AES");

Cipher cipher = Cipher.getInstance("AES");

cipher.init(Cipher.DECRYPT_MODE, key);

byte[] decoded = Base64.getDecoder().decode(encryptedData);

return new String(cipher.doFinal(decoded));
}
}
```

Explanation

This class performs **AES encryption and decryption**.

Functions:

Method	Purpose
encrypt	encrypts PII data
decrypt	decrypts stored data

8. Step 4: User Controller

File:

controller/UserController.java

```
package com.example.securepii.controller;

import com.example.securepii.model.User;
import com.example.securepii.util.EncryptionUtil;

import org.springframework.web.bind.annotation.*;

@RestController
public class UserController {

    @PostMapping("/saveUser")
    public User saveUser(@RequestBody User user) throws Exception {

        user.setEmail(EncryptionUtil.encrypt(user.getEmail()));
        user.setPhone(EncryptionUtil.encrypt(user.getPhone()));

        return user;
    }

    @PostMapping("/getUser")
    public User getUser(@RequestBody User user) throws Exception {

        user.setEmail(EncryptionUtil.decrypt(user.getEmail()));
        user.setPhone(EncryptionUtil.decrypt(user.getPhone()));
    }
}
```



```
    return user;
  }
}
```

Explanation

Two APIs are created:

API	Function
/saveUser	encrypt PII before storing
/getUser	decrypt stored PII

This ensures **data privacy**.

9. Step 5: Run the Application

Open terminal in VS Code.

Run:

```
mvn spring-boot:run
```

Server starts at:

```
http://localhost:8080
```

10. Testing Using Postman

Open **Postman**.

API 1: Encrypt Data

POST

http://localhost:8080/saveUser

Body JSON

```
{  
  "name": "Rahul",  
  "email": "rahul@email.com",  
  "phone": "9876543210"  
}
```

Response (Encrypted Data)

```
{  
  "name": "Rahul",  
  "email": "c29tZUVuY3J5cHRlZERhdGE=",  
  "phone": "YWJjZGVmZ2g=",  
}
```

API 2: Decrypt Data

POST

http://localhost:8080/getUser

Body

```
{  
  "name": "Rahul",  
  "email": "c29tZUVuY3J5cHRlZERhdGE=",  
  "phone": "YWJjZGVmZ2g=",  
}
```

Response

```
{  
  "name": "Rahul",  
  "email": "rahul@email.com",  
  "phone": "9876543210",  
}
```

11. Expected Result

- PII data such as email and phone number are encrypted before storage.
- Encrypted data is decrypted when retrieved.
- Data privacy is maintained during storage and transmission.

12. Conclusion

This experiment demonstrated **secure handling of PII data using AES encryption** in a Java REST API, ensuring protection of sensitive information both **at rest and in transit**.